

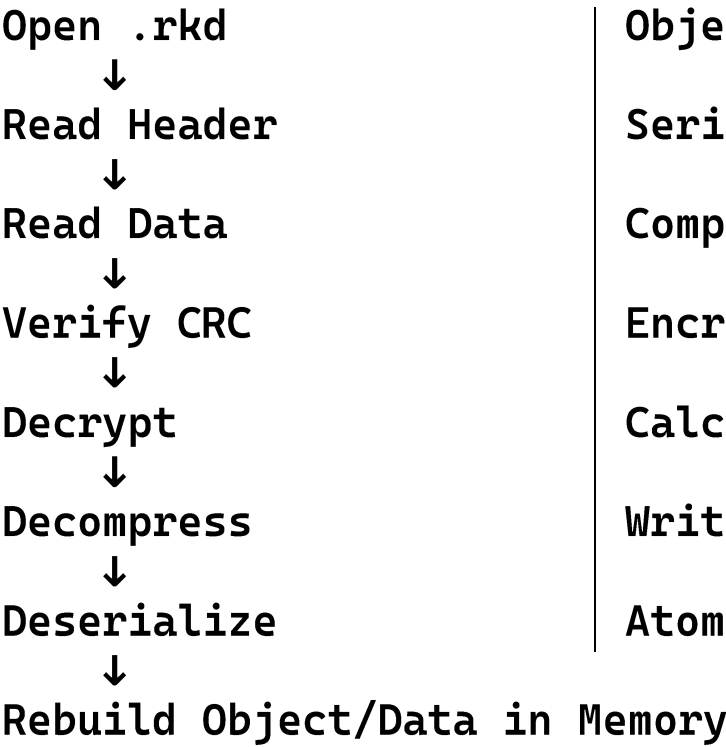
array	linkedlist	doublylist	stack	queue
header magic RKD1 version 1 ds array elem 5 10 20 30 40 50	header magic RKD1 version 1 ds linkedlist elem 4 head 0 0 10 1 1 20 2 2 30 3 3 40 null	header magic RKD1 version 1 ds doublylist elem 4 head 0 0 null 10 1 1 0 20 2 2 1 30 3 3 2 40 null	header magic RKD1 version 1 ds stack elem 5 10 20 30 40 50	header magic RKD1 version 1 ds queue elem 5 10 20 30 40 50

b tree	general tree	binary tree	binary search tree
header magic RKD1 version 1 ds btree elem 4 root 0 0 null 20 40 1 2 3 1 0 10 null 2 0 30 null 3 0 50 null	header magic RKD1 version 1 ds tree elem 7 root 0 0 null A 1 2 3 1 0 B 4 5 2 0 C null 3 0 D 6 4 1 E null 5 1 F null 6 3 G null	header magic RKD1 version 1 ds bitree elem 7 root 0 0 null 10 1 2 1 0 5 3 4 2 0 20 5 6 3 1 3 null null 4 1 7 null null 5 2 15 null null 6 2 30 null null	header magic RKD1 version 1 ds bst elem 7 10 5 20 3 7 15 30

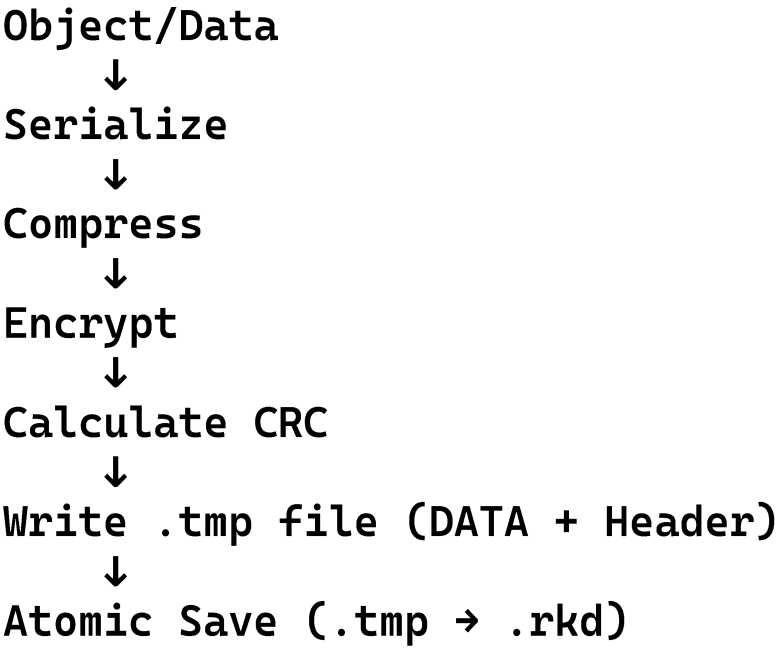
b+ tree	trie	hashtable	disjointset
header magic RKD1 version 1 ds bplustree elem 4 root 0 0 null 30 50 1 2 3 null 1 0 10 20 null 2 2 0 30 40 null 3 3 0 50 60 null null	header magic RKD1 version 1 ds trie elem 3 root 0 0 root 1 4 1 c 2 2 a 3 6 3 t null 4 d 5 5 o 7 6 r null 7 g null	header magic RKD1 version 1 ds hash elem 3 apple 100 banana 200 orange 300	header magic RKD1 version 1 ds djset elem 6 0 0 2 1 0 0 2 0 0 3 3 1 4 3 0 5 5 0 Set 1: 0(rank=2) / \ 1 2 Set 2: 3(rank=1) 4 Set 3: 5(rank=0) node parent rank

graph	directed graph	weighted graph	weighted digraph
header magic RKD1 version 1 ds graph elem 4 0 A 1 2 1 B 0 2 3 2 C 0 1 3 3 D 1 2	header magic RKD1 version 1 ds digraph elem 4 0 A 1 2 1 B 3 2 C 3 3 D null	header magic RKD1 version 1 ds wgraph elem 4 0 A 1:5 2:3 1 B 0:5 3:7 2 C 0:3 3:2 3 D 1:7 2:2	header magic RKD1 version 1 ds wdigraph elem 4 0 A 1:5 2:3 1 B 2:2 3:7 2 C 3:1 3 D null

READ



WRITE



header

magic RKD1
version 1
ds bplustree
dt int
elem 4
root 0

features

checksum crc64
compression gzip
encryption aes256
~~record variable~~
~~freespace on~~
cache lru
~~transaction wal~~

data

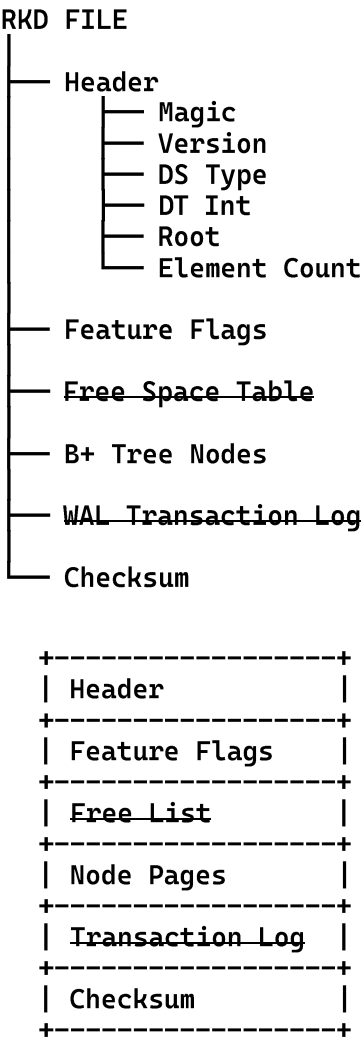
0	null	30	50		1	2	3		null
1	0	10	20		null				2
2	0	30	40		null				3
3	0	50	60		null				null

~~freespace~~

128	32
256	64

checksum

0x995DC9BBDF1939FA



- Linear
- Array
 - Linked List
 - Doubly Linked List
 - Circular Linked List
 - Stack
 - Queue
 - Deque
- Trees
- Binary Tree
 - Binary Search Tree (BST)
 - AVL Tree
 - Red-Black Tree
 - General Tree
 - B-Tree
 - B+ Tree
 - Trie
 - Heap (Min Heap / Max Heap)
 - Segment Tree
 - Fenwick Tree (Binary Indexed Tree)
- Graphs
- Undirected Graph
 - Directed Graph
 - Weighted Graph
 - DAG (Directed Acyclic Graph)
- Hashing
- Hash Table
 - Hash Map
 - Hash Set
- Sets
- Disjoint Set (Union-Find)

```
import 'dart:io';
```

```
File file = File("data.rkd");  
List<int> bytes = file.readAsBytesSync();
```

```
File file = File("data.rkd");  
file.writeAsBytesSync(bytes);
```

🔍 Checksum

```
import 'package:crc64/crc64.dart';  
  
int crc = crc64(bytes);
```

```
dependencies:  
  crc64: ^1.0.0
```

```
File temp = File("data.tmp");  
temp.renameSync("data.rkd");
```

📦 Compress

📁 Decompress

```
List<int> compress(List<int> data) {  
  return GZipCodec().encode(data);  
}  
  
List<int> decompress(List<int> data) {  
  return GZipCodec().decode(data);  
}
```

```
import 'dart:io';  
import 'dart:convert';  
  
List<int> compressed = compress(data);  
  
List<int> compressed = file.readAsBytesSync();  
List<int> original = decompress(compressed);
```

dependencies:

```
  cryptography: ^2.7.0
```

```
import 'package:cryptography/cryptography.dart';
```

🔒 AES-256 Encrypt

```
Future<List<int>> encryptData(List<int> data, SecretKey key) async {  
  final secretBox = await algorithm.encrypt(  
    data,  
    secretKey: key,  
  );  
  
  return secretBox.concatenation();  
}
```

🔓 Decrypt

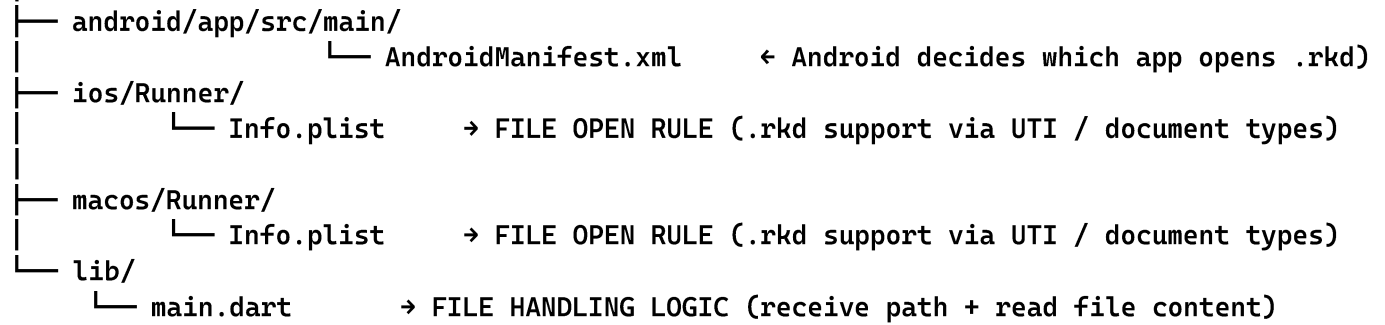
```
Future<List<int>> decryptData(List<int> encrypted, SecretKey key) async {  
  final secretBox = SecretBox.fromConcatenation(  
    encrypted,  
    nonceLength: 12,  
    macLength: 16,  
  );  
  
  return await algorithm.decrypt(  
    secretBox,  
    secretKey: key,  
  );  
}
```

```
final algorithm = AesGcm.with256bits();  final key = await algorithm.newSecretKey();
```

SecretBox → A container that holds all three: CipherText + Nonce + MAC.

- CipherText → The actual encrypted data (original data turned into unreadable bytes).
- Nonce → A random unique value used during encryption so the same data doesn't produce the same encrypted output every time. 12 bytes (default)
- MAC (Message Authentication Code) → A security tag that verifies the encrypted data hasn't been modified or corrupted. 16 bytes (default)

Flutter Project



```
import 'dart:io';
import 'package:receive_sharing_intent/receive_sharing_intent.dart';

void initSharedFiles() {
  // Get files when app is opened from outside (share/open intent)
  ReceiveSharingIntent.instance.getInitialMedia().then((files) async {

    final rkdFiles = files.where((f) { // filter only .rkd files & remove fake ones
      return f.path.endsWith('.rkd') &&
        (f.mimeType == 'application/octet-stream' ||
          f.mimeType == null); // can be rename file.jpg → file.rkd
    }).toList();

    if (rkdFiles.isEmpty) { print('No .rkd file found'); return; }
    for (final file in rkdFiles)
      print('REAL RKD file: ${file.path}');

    // read first valid file
    final file = File(rkdFiles.first.path); // Take first valid .rkd file
    final content = await file.readAsString(); // Read file content as text
    print(content);
  });
}
```

android

```
<activity
  android:name=".MainActivity"
  android:exported="true">

  <intent-filter>
    <action android:name="android.intent.action.MAIN"/>
    <category android:name="android.intent.category.LAUNCHER"/>
  </intent-filter>

  <!-- Add here -->
  <intent-filter>
    <action android:name="android.intent.action.VIEW"/>
    <category android:name="android.intent.category.DEFAULT"/>
    <data
      android:scheme="file"
      android:pathPattern=".*\\.rkd"/>
  </intent-filter>

</activity>
```

```
<key>CFBundleDocumentTypes</key>
<array>
  <dict>
    <key>CFBundleTypeName</key>
    <string>RKD File</string>

    <key>LSItemContentTypes</key>
    <array>
      <string>public.data</string>
    </array>

    <key>CFBundleTypeRole</key>
    <string>Editor</string>
  </dict>
</array>
```

```
<key>CFBundleDocumentTypes</key>
<array>
  <dict>
    <key>CFBundleTypeName</key>
    <string>RKD File</string>

    <key>LSItemContentTypes</key>
    <array>
      <string>public.data</string>
    </array>

    <key>CFBundleTypeRole</key>
    <string>Editor</string>
  </dict>
</array>
```

 windows

✓ If you are building installer:

👉 Write it in installer script (Inno Setup / MSIX / EXE installer)

✓ If manual setup: 👉 Write it in a .reg file (Windows Registry file)

[Registry]

```
Root: HKCR; Subkey: ".rkd"; ValueType: string; ValueData: "RKDFile"
Root: HKCR; Subkey: "RKDFile"; ValueType: string; ValueData: "RKD File"
Root: HKCR; Subkey: "RKDFile\shell\open\command"; ValueType: string;
ValueData: ""{app}\your_app.exe"" "%1"""
```

linux

1. Linux does NOT have one installer system

Unlike Windows (Inno Setup style), Linux uses multiple package systems:

- .deb → Debian / Ubuntu
- Flatpak → universal app format
- .rpm → Fedora / RedHat
- Snap → Ubuntu ecosystem

👉 Each system handles installation differently.

2. File association is NOT part of installer only

Linux uses desktop + MIME system:

📄 Desktop file [my_app.desktop] + 📄 MIME file [x-rkd.xml]

3. Two installation locations

🟢 System-wide (all users)
 /usr/share/applications/my_app.desktop
 /usr/share/mime/packages/x-rkd.xml
 ✓ Needs sudo (admin permission)
 ✓ Used when app is installed globally

🟡 User-only (no root needed)

~/.local/share/applications/my_app.desktop
 ~/.local/share/mime/packages/x-rkd.xml
 ✓ Used in development
 ✓ Most Flutter desktop apps use this

📄 4. Desktop file content

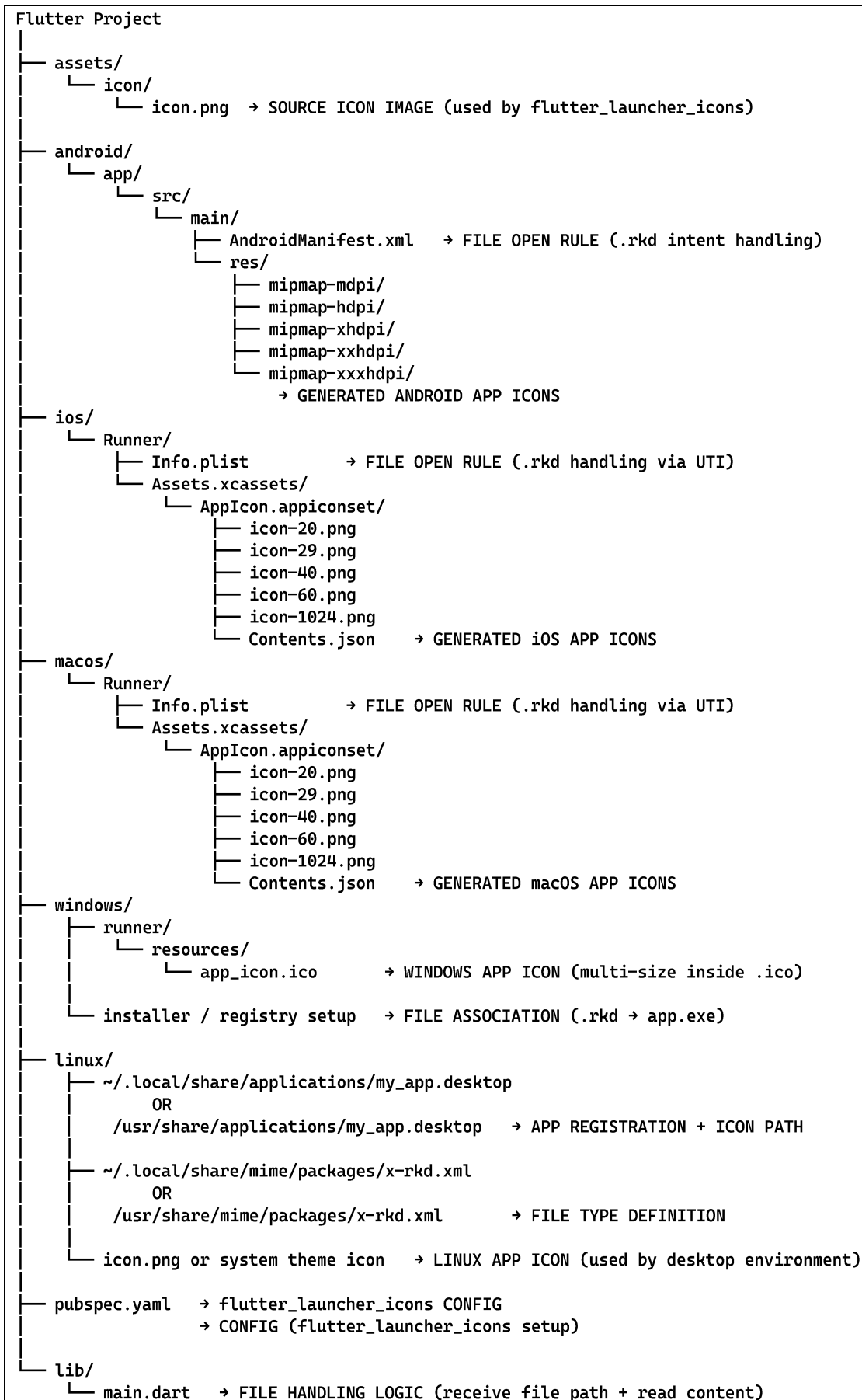
```
[Desktop Entry]
Name=MyApp
Exec=/path/to/your_app %f
Type=Application
MimeType=application/x-rkd;
```

📄 5. MIME file content

```
<mime-type type="application/x-rkd">
  <comment>RKD File</comment>
  <glob pattern="*.rkd"/>
</mime-type>
```

💡 6. Simple rule

```
System install →
  /usr/share/...
User install →
  ~/.local/share/...
```



Android → icon.png → flutter_launcher_icons → res/mipmap-* → app launcher icon

iOS → icon.png → flutter_launcher_icons → AppIcon.appiconset → app icon

macOS → icon.png → flutter_launcher_icons → AppIcon.appiconset → dock/app icon

Windows → icon.ico → build/installer → embedded into .exe → app icon (start/taskbar)

Linux → icon.png → .desktop file → system icon theme → app launcher icon

Here's the real, practical way to set your app/file icon on Windows + Linux (for Flutter desktop apps).

Windows – App Icon + File Association

🔥 1. App icon (EXE icon)

✓ You set it in Flutter build:

File:

windows/runner/resources/app_icon.ico

👉 Replace app_icon.ico with your icon

✓ Then rebuild:

flutter build windows

👉 Your .exe will now show that icon in:

- Start menu
 - Taskbar
 - Desktop shortcut
-

🔥 2. File icon (.rkd association)

Windows uses registry/installer (NOT Flutter code)

Example (Inno Setup / installer):

[Registry]

Root: HKCR; Subkey: ".rkd"; ValueData: "RKDFile"

Root: HKCR; Subkey: "RKDFile\DefaultIcon";

ValueData: "C:\\Program Files\\MyApp\\app.exe,0"

Root: HKCR; Subkey: "RKDFile\\shell\\open\\command";

ValueData: "\"C:\\Program Files\\MyApp\\app.exe\" \"%1\""

🧠 Result:

.rkd file → your app icon shows in file explorer + opens your app

Linux – App Icon + File Association

Linux uses .desktop file system

🔥 1. App icon

File:

/usr/share/icons/hicolor/256x256/apps/myapp.png

or user:

~/.local/share/icons/myapp.png

🔥 2. Desktop file

~/.local/share/applications/my_app.desktop

Content:

[Desktop Entry]

Name=MyApp

Exec=/path/to/your_app %f

Icon=/path/to/myapp.png

Type=Application

MimeType=application/x-rkd;

🔥 3. MIME type (file association)

~/.local/share/mime/packages/x-rkd.xml

<mime-type type="application/x-rkd">

<comment>RKD File</comment>

<glob pattern="*.rkd"/>

</mime-type>

🧠 Result:

.rkd file → shows icon (from .desktop file) → opens your app

⚡ FINAL SIMPLE SUMMARY

🖥 Windows

- App icon → .ico in runner/resources
- File icon → registry/installer

🐧 Linux

- App icon → .png via .desktop file
 - File icon → MIME + desktop entry
-

🔥 ONE LINE MEMORY

Windows → .ico + registry

Linux → .desktop + icon + MIME

If you want, I can next show:

👉 how to make .rkd file show your custom icon in file manager preview (advanced trick) 🚀